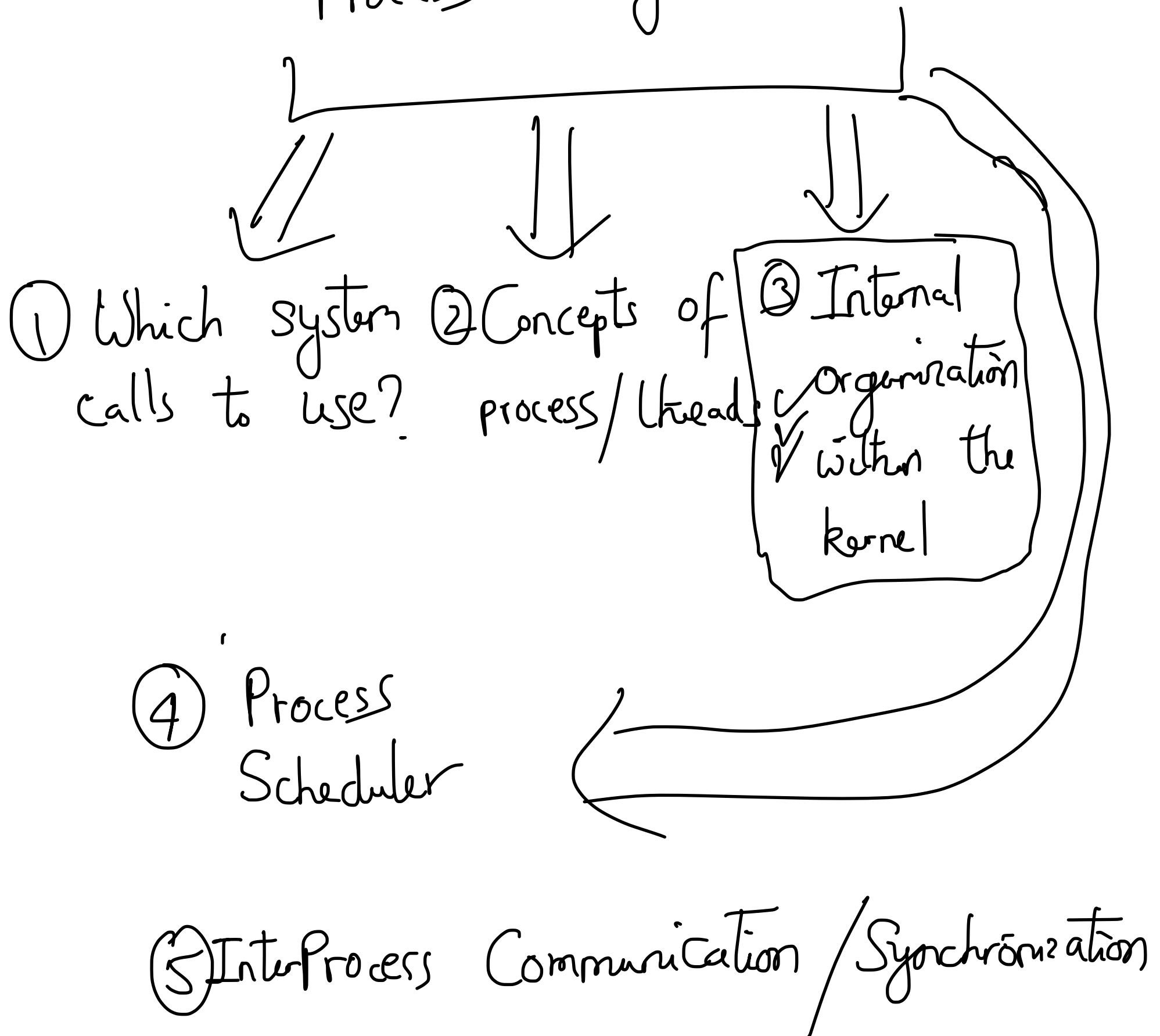
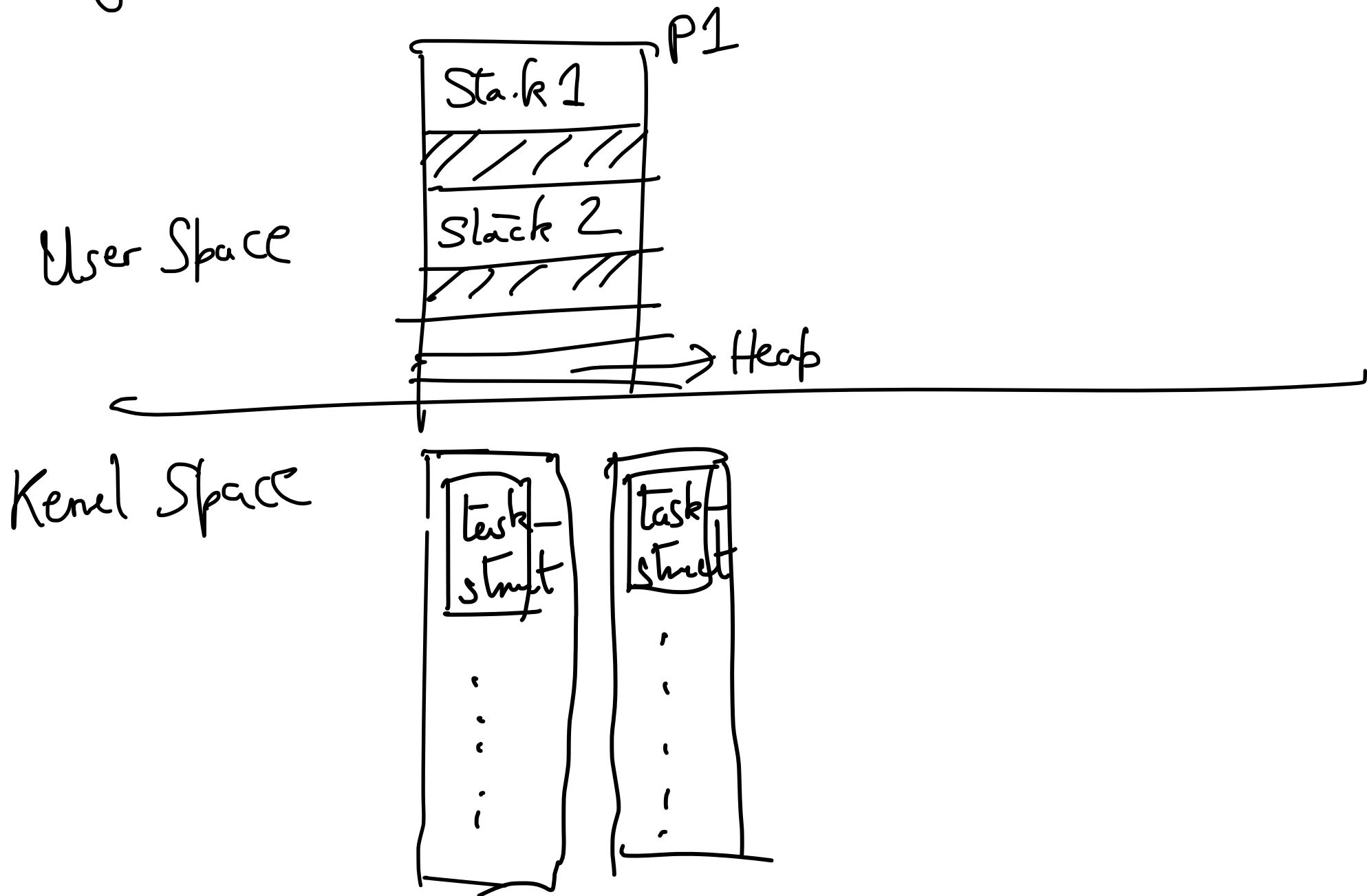


Assembly Programming

Process Management



Address Space : $\rightarrow$ Memory area where
you store the process related data.



Internal data structures related to each process ~~is~~ kept by the kernel. This is kept in the process ~~space~~ AS within the

kernel space.

In Linux, process control block is called task_struct.

You can enter into (execute code from) kernel space in two ways:

→ 1) Process Context → Enter into it by using system calls

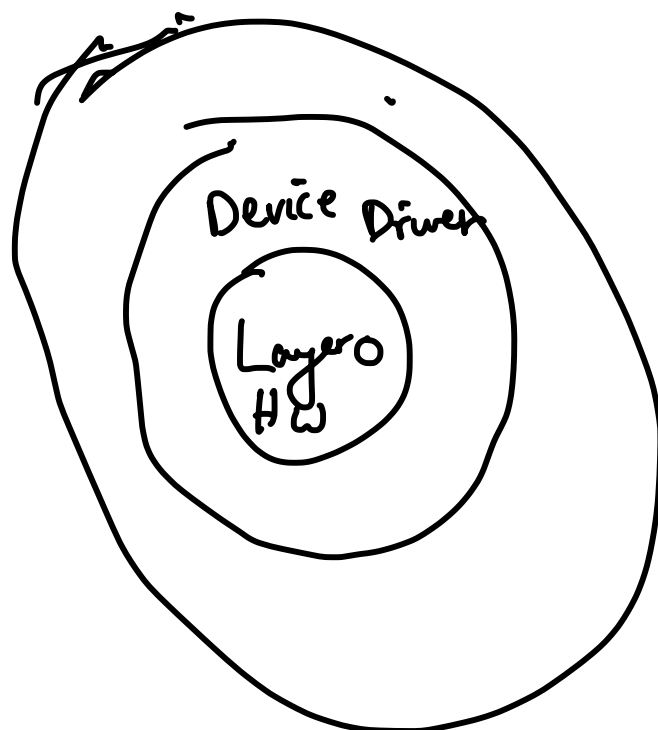
2) Interrupt Context → Enters into using hardware signals.

How does the kernel handle requests?

→ Linux is a monolithic kernel.

(Any function can call any other function within the kernel).

→ Layered operating system (UNIX System V, Berkeley UNIX)



→ Microkernels → Move the file system and device drivers to the user space. The OS is secure from crashes due to device driver bugs.

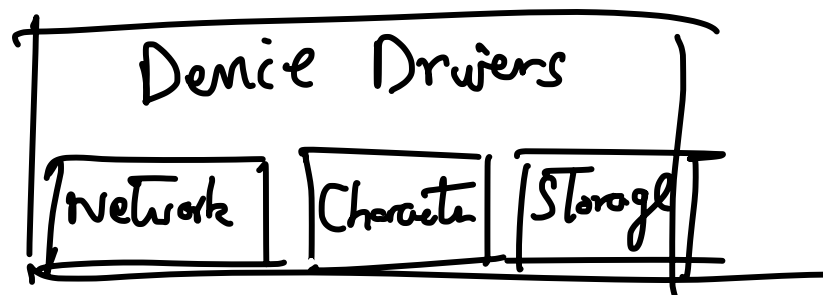
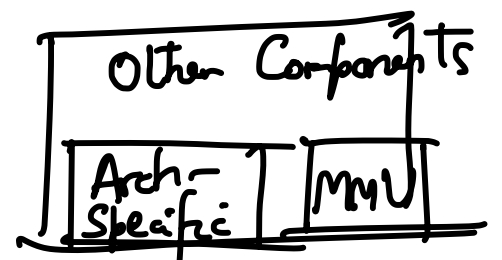
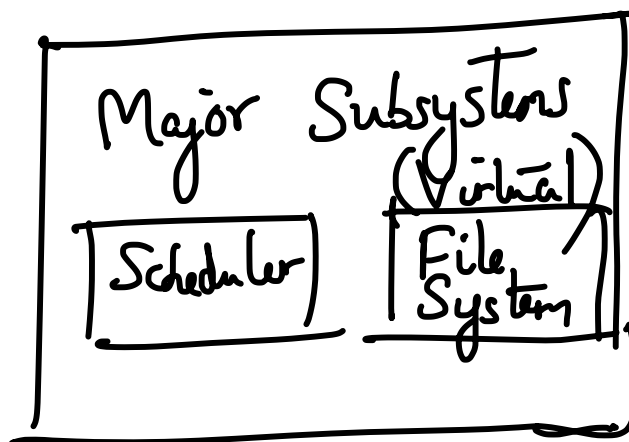
⇒ Very reliable; used in embedded and safety-critical systems.

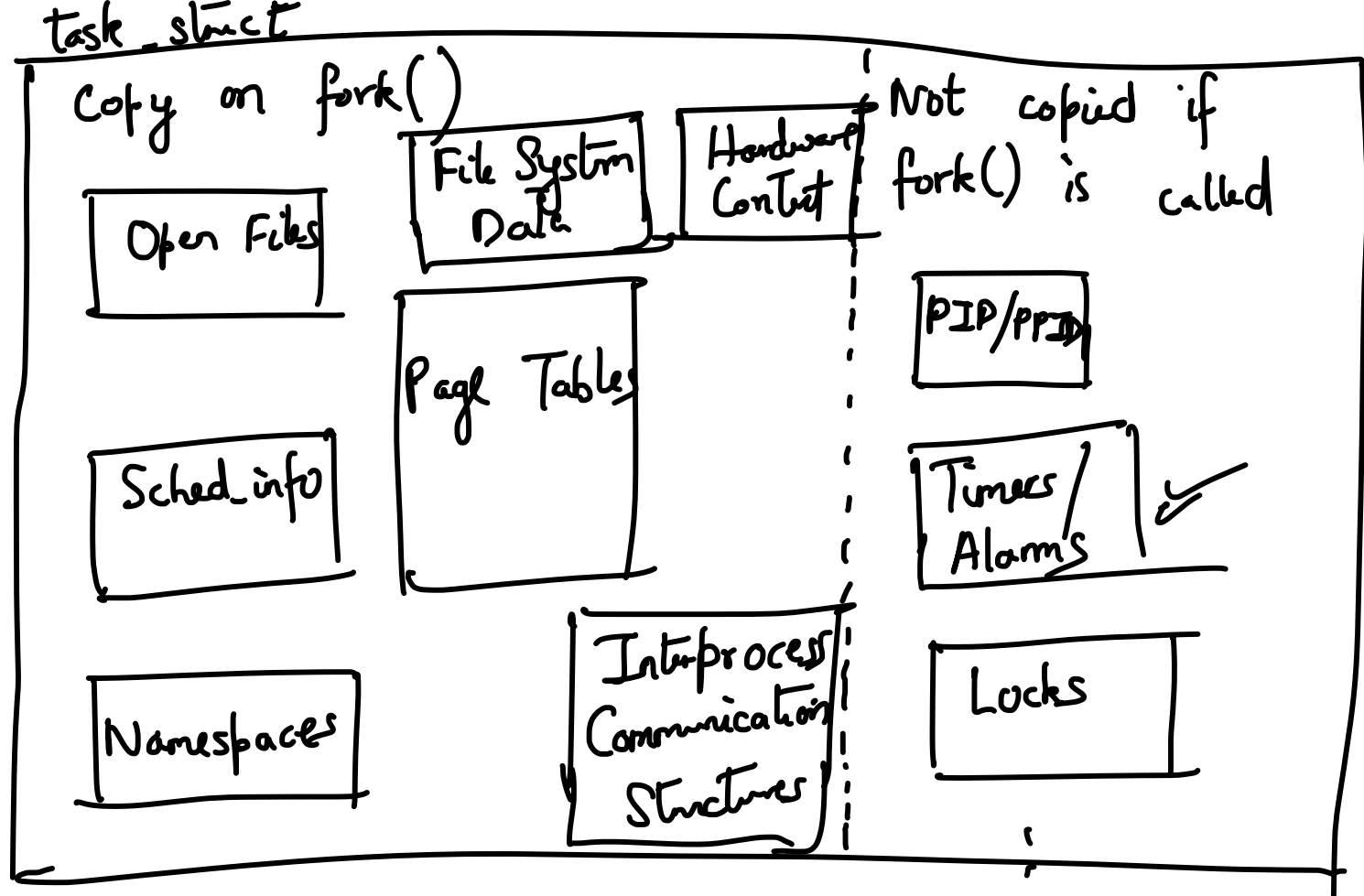
GNU Hurd → Attempt to create a microkernel for desktop systems.

Linux Kernel Modules: → Parts of the kernel that are not needed to run the basic code of processes.

Modules can be loaded when the kernel is running. They can also be removed if it is not used. Once loaded, the modules can access any Linux function.

Kernel Space
(Common)





Circular linked lists to keep track of task_structs. There is a "current" variable which keeps track of the current task_struct. This current variable is stored in a register. Whenever the kernel wants to make a ~~switch~~ process switch, the current variable has to be modified. Terminating a process for a kernel is same as destroying the task_struct.

A kernel module that prints "Hello world". Load it into the kernel and see whether the message comes up.